

SDE interview guide

For final-year students targeting Software Developer roles: how campus hiring works, a 4-week plan, DSA and CS fundamentals, solving problems live, resume talking points and common questions.

1. How campus hiring works

Most campus drives for developer roles follow the same three stages: an online test, one or more technical interviews, and an HR interview. What changes between companies is how hard each stage is and what it focuses on.

Service companies (for example TCS, Infosys, Wipro, Cognizant, Accenture)

- **Online test:** aptitude (quantitative), logical reasoning, verbal English, and usually one or two coding questions of easy to medium level. Some companies also have an essay or email-writing section.
- **Technical interview:** questions on your resume and projects, one programming language in depth, OOP, DBMS/SQL, basic OS and networks. You may be asked to write a short program on paper or in a shared editor (reverse a string, check a palindrome, find the second largest number).
- **HR interview:** communication, attitude, willingness to relocate or work in shifts, and sometimes questions about a service agreement or bond. Sometimes the technical and HR rounds are merged into one.
- **Higher-pay tracks:** many service companies also run a separate, tougher track for "digital" or "specialist" roles with harder coding questions. If you clear the basic test well, you may be offered this track.

Product companies and startups

- **Online assessment (OA):** usually 2 to 4 coding problems in 60 to 120 minutes on an online judge. Some add MCQs on CS fundamentals or debugging. Only hidden test cases matter, so edge cases and time limits are important.
- **Technical rounds (often 2 to 4):** data structures and algorithms on a shared editor or whiteboard, explained out loud. Later rounds may include basic low-level design (classes for a parking lot, a library system) or project deep-dives.
- **Hiring manager / HR round:** behavioural questions, motivation, teamwork, and how you handle failure or disagreement.

Stage	Service companies	Product companies
Online test	Aptitude + reasoning + verbal + easy/medium coding	Mainly DSA coding, medium to hard
Technical	Resume, language basics, OOP, SQL, simple programs	DSA in depth, complexity analysis, sometimes design
HR	Communication, relocation, shifts, fit	Behavioural, motivation, culture fit
What wins	Accuracy, speed, clear basics, confidence	Problem-solving process, clean code, edge cases

Tip: Prepare for both. Aptitude and CS basics are needed even for product companies (many OAs include MCQs), and DSA practice makes service-company coding sections easy.

2. A 4-week prep plan

This plan assumes 3 to 4 hours a day alongside college. If you have more time, add more practice problems, not more topics. Pick **one** language (C++, Java or Python) and use it for everything.

Week	DSA (about 2 hours/day)	CS fundamentals and aptitude (about 1 hour/day)	Other
1	Arrays, strings, hashing (maps and sets), two pointers, prefix sums. 4 to 5 easy problems a day.	OOP: four pillars, overloading vs overriding, abstract class vs interface. Aptitude: percentages, ratios, time and work.	Rewrite your resume. List 2 projects you can explain fully.
2	Sliding window, binary search, sorting, recursion, stacks and queues. Mix easy and medium.	DBMS: keys, normalization, joins, ACID, indexing. Practise 15 to 20 SQL queries.	Prepare "Tell me about yourself" and project explanations. Say them out loud.
3	Linked lists, trees (traversals, BST), heaps, basic graphs (BFS, DFS). Mostly medium.	OS: processes vs threads, scheduling, deadlock, memory, paging. Networks: OSI/TCP-IP, TCP vs UDP, HTTP, DNS.	One timed mock online test (aptitude + coding).
4	Basic dynamic programming (1D, simple 2D), greedy. Revise every problem you got wrong earlier.	Revise all fundamentals using short notes. Aptitude: probability, permutations, number system.	2 to 3 mock interviews with friends. Prepare behavioural stories (STAR).

Tip: Keep an "error notebook". For every problem you could not solve, write one line: the key idea you missed. Read this notebook before every interview.

3. DSA topics by priority

Do not try to cover everything. Topics at the top appear far more often in campus tests and interviews than those at the bottom.

Priority	Topics	Must-know patterns
Must do	Arrays, strings, hashing	Frequency counting, two pointers, sliding window, prefix sum, Kadane's algorithm (maximum subarray sum)
Must do	Sorting and searching	Binary search on a sorted array, binary search on the answer, merge sort, custom comparators
Must do	Stacks and queues	Balanced brackets, next greater element (monotonic stack), queue using two stacks
Must do	Linked lists	Reverse a list, find the middle (slow/fast pointers), detect a cycle, merge two sorted lists
High	Trees and BST	Inorder/preorder/postorder, level-order (BFS), height, lowest common ancestor, validate a BST
High	Recursion and backtracking	Subsets, permutations, N-Queens idea
Medium	Heaps	Top-k elements, k-th largest, merge k sorted lists
Medium	Graphs	BFS, DFS, number of islands, cycle detection, topological sort, shortest path in an unweighted graph
Medium	Dynamic programming	Climbing stairs, house robber, 0/1 knapsack, longest common subsequence, coin change
Lower	Advanced	Tries, union-find (disjoint set), Dijkstra, segment trees (mainly for top product companies)

Complexities you must know without thinking

Operation	Time
Array access by index	$O(1)$
Search in an unsorted array	$O(n)$
Binary search (sorted array)	$O(\log n)$
Hash map insert / lookup	$O(1)$ average, $O(n)$ worst case
Merge sort	$O(n \log n)$ in all cases, $O(n)$ extra space
Quick sort	$O(n \log n)$ average, $O(n^2)$ worst case
Heap insert / remove top	$O(\log n)$; build a heap from n items: $O(n)$
Balanced BST search / insert	$O(\log n)$; unbalanced BST worst case $O(n)$
BFS / DFS on a graph	$O(V + E)$ with an adjacency list

Tip: A rough rule for online judges: about 10^8 simple operations run in around one second. If n is up to 10^5 , an $O(n^2)$ solution (10^{10} operations) will usually time out; aim for $O(n \log n)$ or $O(n)$.

4. CS fundamentals checklist

Interviewers rarely go very deep here for freshers, but they expect clear, correct definitions with one example each. Tick these off.

OOP

- **Four pillars:** encapsulation (bundle data with methods, hide internal state), abstraction (show only what is needed), inheritance (reuse a parent class), polymorphism (one interface, many forms).
- **Overloading vs overriding:** overloading = same method name, different parameters, decided at compile time. Overriding = child class redefines a parent method with the same signature, decided at run time.
- **Abstract class vs interface** (Java view): an abstract class can have state (fields) and constructors; a class can extend only one class but implement many interfaces.
- Constructors, `this` / `super`, access modifiers, static vs instance members.
- Composition vs inheritance ("has-a" vs "is-a"). Prefer composition when the relationship is not truly "is-a".
- Basic awareness of SOLID principles and one design pattern you can explain (Singleton or Factory).

DBMS and SQL

- Primary key, foreign key, candidate key, unique key, composite key.
- **Normalization:** 1NF (atomic values, no repeating groups), 2NF (1NF + no partial dependency on part of a composite key), 3NF (2NF + no transitive dependency), BCNF (every determinant is a candidate key).

- **ACID:** Atomicity (all or nothing), Consistency (valid state to valid state), Isolation (concurrent transactions do not interfere), Durability (committed data survives crashes).
- Joins: INNER, LEFT, RIGHT, FULL OUTER, self join. WHERE vs HAVING ; GROUP BY ; DELETE vs TRUNCATE vs DROP .
- Indexes: speed up reads, slow down writes, take extra space. Usually implemented with B-trees / B+ trees.
- SQL vs NoSQL: fixed schema and joins vs flexible schema and easy horizontal scaling.

A query that is commonly asked: the second highest salary.

```
SELECT MAX(salary) AS second_highest
FROM employee
WHERE salary < (SELECT MAX(salary) FROM employee);
```

Operating systems

- **Process vs thread:** a process has its own memory space; threads inside a process share memory, so they are cheaper to create and switch but need synchronisation.
- CPU scheduling: FCFS, SJF, Round Robin, Priority. Know what preemptive means.
- **Deadlock** needs all four conditions: mutual exclusion, hold and wait, no preemption, circular wait. Breaking any one prevents it.
- Mutex vs semaphore; race condition; critical section.
- Virtual memory, paging, page fault, thrashing; stack vs heap memory.

Computer networks

- OSI model (7 layers: Physical, Data Link, Network, Transport, Session, Presentation, Application) and the TCP/IP model.
- **TCP vs UDP:** TCP is connection-oriented, reliable and ordered (three-way handshake: SYN, SYN-ACK, ACK). UDP is connectionless and faster, with no delivery guarantee (used for video calls, DNS queries, games).
- HTTP vs HTTPS (HTTPS = HTTP over TLS; default ports 80 and 443). Common status codes: 200, 201, 301, 400, 401, 403, 404, 500.
- DNS, IP address vs MAC address, what a router and switch do.
- "What happens when you type a URL in the browser?" (answer in section 7).

5. How to approach a coding problem in an interview

Interviewers judge your **process** as much as your final code. Talk through these six steps every time.

1. **Clarify.** Repeat the problem in your own words. Ask about input size, value ranges, empty input, duplicates, negative numbers, and the expected output format.
2. **Examples.** Work through one normal example and one edge case by hand. This often reveals the pattern.
3. **Brute force.** State the simplest correct solution and its complexity, even if it is slow. This shows you can solve it at all.

4. **Optimise.** Ask: what work is repeated? Can a hash map, sorting, two pointers or a sliding window remove it? Explain the idea before coding.
5. **Code and test.** Write clean code with clear names. Then dry-run it on your examples line by line, including edge cases.
6. **Complexity.** State the final time and space complexity and explain why.

Worked example: longest substring without repeating characters

Problem: given a string `s`, return the length of the longest substring (contiguous part) that has no repeated characters.

1. Clarify. "Substring means contiguous, right? Can the string be empty? Which characters can appear?"
Assume: contiguous, may be empty (answer 0), any characters.

2. Examples. `"abcabcbb"` gives 3 (`"abc"`). `"bbbbbb"` gives 1. `"pwwkew"` gives 3 (`"wke"` ; note `"pwke"` is not contiguous). `""` gives 0.

3. Brute force. Start at every index and extend to the right until a character repeats.

```
def longest_brute(s):
    best = 0
    for i in range(len(s)):
        seen = set()
        for j in range(i, len(s)):
            if s[j] in seen:
                break
            seen.add(s[j])
            best = max(best, j - i + 1)
    return best
```

Time $O(n^2)$ in the worst case, space $O(k)$ where k is the number of distinct characters. Fine for small inputs, too slow for $n = 10^5$.

4. Optimise. When we find a repeat, we do not need to restart from scratch. Keep a window `[start, i]` with no repeats and remember the last index of each character. If the current character was last seen inside the window, move `start` just past that position. Each index is visited once: a sliding window.

```
def length_of_longest_substring(s):
    last_seen = {} # character -> last index where it appeared
    start = 0      # left edge of the current window
    best = 0
    for i, ch in enumerate(s):
        if ch in last_seen and last_seen[ch] >= start:
            start = last_seen[ch] + 1
        last_seen[ch] = i
        best = max(best, i - start + 1)
    return best

tests = {"abcabcbb": 3, "bbbbbb": 1, "pwwkew": 3, "": 0, "abba": 2, " ": 1}
for text, expected in tests.items():
    assert length_of_longest_substring(text) == expected
    assert longest_brute(text) == expected
print("all tests passed")
```

5. Test. Dry-run "abba", which catches a common bug:

i	ch	Action	start	best
0	a	new character	0	1
1	b	new character	0	2
2	b	seen at 1, inside window, start = 2	2	2
3	a	seen at 0, but 0 < start, so ignore	2	2

Without the check `last_seen[ch] >= start`, step 3 would move `start` back to 1 and wrongly return 3.

6. Complexity. Time $O(n)$ because each character is processed once with $O(1)$ average dictionary operations. Space $O(k)$ for the dictionary, where k is the number of distinct characters.

Tip: If you get stuck, say what you are thinking: "I am looking for repeated work in the brute force." Interviewers often give hints, but only if they can hear your reasoning.

6. Resume and project talking points

Resume rules

- One page. Order: contact details, education, skills, projects, internships/experience, achievements.
- Only list skills you can answer questions on. If you write "Java, SQL, React", expect questions on all three.
- Write project points as **action + what + result**: "Built a REST API in Node.js for event registration with JWT login and input validation" is better than "Worked on backend".
- Use numbers only if they are true and you can explain them (users, response time, dataset size).
- Add links to GitHub or a live demo, and make sure they work and the code is readable.
- Check spelling, consistent dates and formatting. Save as PDF.

Be ready to explain every project

For each project on your resume, prepare short answers to these questions:

1. What problem does it solve and who uses it?
2. What is the architecture? (Frontend, backend, database, how they talk to each other.) Be able to draw it.
3. Why did you choose this tech stack over the alternatives?
4. What was the hardest bug or challenge, and how did you fix it?
5. What exactly was **your** part, if it was a team project?
6. How is the database designed? (Main tables and relationships.)
7. What would you improve or add with more time? How would it handle 10 times more users?

Tip: Interviewers often pick one line from your resume and go deep. If you cannot explain a project for five minutes, remove it or study it again.

7. Common technical questions with short answers

Q. What is the difference between an array and a linked list?

An array stores elements in contiguous memory, so access by index is $O(1)$, but inserting or deleting in the middle is $O(n)$ because elements shift. A linked list stores nodes connected by pointers, so insertion or deletion at a known node is $O(1)$, but access by position is $O(n)$. Arrays are also more cache-friendly.

Q. How does a hash map work?

A hash function converts the key into an index of an internal array (bucket). Different keys can map to the same bucket (a collision), handled by chaining (a list per bucket) or open addressing (probing for another slot). When the table gets too full, it is resized and entries are rehashed. Average lookup is $O(1)$; worst case is $O(n)$ if many keys collide.

Q. Stack vs queue?

A stack is LIFO (last in, first out), used for function calls, undo and bracket matching. A queue is FIFO (first in, first out), used for BFS, task scheduling and buffers.

Q. What is polymorphism? Give an example.

The same interface behaving differently for different types. Compile-time polymorphism is method overloading; run-time polymorphism is method overriding. Example: a `Shape` class with an `area()` method that `Circle` and `Rectangle` override; calling `shape.area()` runs the correct version for the actual object.

Q. Process vs thread?

A process is a running program with its own memory space. Threads are units of execution inside a process that share its memory. Threads are lighter to create and switch, but shared memory means you need locks to avoid race conditions. One process crashing usually does not affect another; one thread crashing can bring down the whole process.

Q. What is a deadlock and how can it be prevented?

Two or more processes each wait for a resource the other holds, so none can continue. It needs mutual exclusion, hold and wait, no preemption and circular wait at the same time. Prevent it by breaking one condition, for example always acquiring locks in a fixed global order (removes circular wait).

Q. What is normalization and why is it used?

Organising tables to reduce duplicate data and avoid insert, update and delete anomalies, by splitting data into related tables (1NF, 2NF, 3NF, BCNF). Sometimes tables are deliberately denormalized to make reads faster.

Q. What is an index? Should we index every column?

An index is a separate data structure (usually a B+ tree) that lets the database find rows without scanning the whole table. No: each index uses extra storage and slows down inserts, updates and deletes. Index columns used often in WHERE, JOIN and ORDER BY.

Q. TCP vs UDP?

TCP sets up a connection, guarantees delivery and order, and controls flow and congestion; used for web pages, email and file transfer. UDP sends packets without a connection or guarantees, so it has lower overhead and delay; used for live video, voice, online games and DNS queries.

Q. What happens when you type a URL in the browser and press Enter?

The browser finds the server's IP address using DNS (checking caches first). It opens a TCP connection (and a TLS handshake for HTTPS), then sends an HTTP request. The server processes it and sends back a response with HTML. The browser parses the HTML, requests CSS, JavaScript and images, builds the page and renders it.

Q. What is the difference between `==` and `equals()` in Java?

For objects, `==` checks whether two references point to the same object. `equals()` checks logical equality as defined by the class (for `String`, same characters). For primitives, `==` compares values.

Q. What is REST?

A style for designing web APIs around resources identified by URLs, using HTTP methods: GET (read), POST (create), PUT/PATCH (update), DELETE (remove). It is stateless: each request carries all the information the server needs, such as an auth token.

8. Common behavioural questions

Use the **STAR** method for "tell me about a time" questions: **S**ituation (one or two lines of context), **T**ask (what you had to do), **A**ction (what *you* did, in detail), **R**esult (outcome and what you learned). Prepare 4 to 5 stories from projects, internships, college events or team work, and reuse them.

Q. Tell me about yourself.

Keep it to about one minute: name and degree, your main technical interest, one or two projects or an internship with what you built, one achievement, and why you are interested in this role. Example: "I am a final-year Computer Science student. I mainly work in Java and SQL. In my last project I built an attendance system with a Spring Boot backend and MySQL, used by our department. I enjoy solving DSA problems and have practised regularly for the last year. I am looking for a developer role where I can work on real products and keep learning."

Q. What are your strengths and weaknesses?

Strength: pick one relevant to the job and prove it with an example ("I kept at a problem: in my project I spent two days tracking a login bug down to a timezone issue"). Weakness: pick a real but fixable one and show what you are doing about it ("I used to hesitate to ask for help early; now I set myself a time limit before asking").

Q. Why do you want to join our company?

Research before the interview: what the company builds, its main clients or products, and the role. Connect it to your interests: "Your company works on large client projects across banking and retail. I want exposure to large production systems early in my career, and the training programme for freshers would help me build strong foundations."

Q. Tell me about a conflict in your team.

Choose a real, mild disagreement. Show that you listened, focused on the goal rather than the person, and reached a decision. "Two of us wanted different databases for our project. We listed our requirements, built a small test of both in an hour, and picked the one that fit better. We finished on time and I learned to settle such debates with data."

Q. Tell me about a failure.

Own it, do not blame others, and spend most of the answer on what you learned and changed. For example, missing a hackathon submission because you did not test the deployment early, and now always deploying a basic version on day one.

Q. Are you willing to relocate or work in shifts?

Answer honestly. If you are flexible, say so clearly. If you have a real constraint, state it politely. Do not say yes if you do not mean it.

Q. Where do you see yourself in five years?

Show growth within the field, not leaving: "I want to become a strong engineer who can own a module end to end, mentor juniors, and take on design responsibilities."

Q. Do you have any questions for us?

Always ask one or two: "What does a new developer's first few months look like?" or "Which technologies does this team use most?" Avoid asking about salary in a technical round.

9. Mistakes to avoid

- **Coding immediately.** Jumping into code without clarifying or explaining your approach. Spend the first few minutes on understanding and planning.
- **Silence.** Thinking quietly for minutes. Speak your thoughts so the interviewer can follow and help.
- **Ignoring edge cases.** Empty input, a single element, duplicates, negative numbers, very large values (integer overflow in C++/Java).
- **Memorising solutions.** Interviewers change the question slightly. Learn patterns and why they work.
- **Resume padding.** Listing technologies or projects you cannot explain. One deep project beats five shallow ones.
- **Guessing confidently.** If you do not know, say "I am not sure, but I think..." and reason it out. Bluffing is easy to detect.
- **Skipping aptitude.** Many candidates with good coding skills fail the first round on aptitude or time management.
- **Too many languages.** Switching languages during preparation. Master one for DSA.
- **Neglecting communication.** Mumbling, very long answers, or speaking badly of a previous team or college.
- **Not testing in online tests.** Submit only after checking sample cases and a couple of your own edge cases. Read the input and output format carefully.

10. Last-week checklist

- Re-read your error notebook and re-solve 10 to 15 problems you previously got wrong.
- Revise complexities of common operations and sorting algorithms.
- Revise one-page notes for OOP, DBMS/SQL, OS and networks. Practise 10 SQL queries.
- Take at least one full timed mock test (aptitude + coding) under exam conditions.
- Say "Tell me about yourself" and each project explanation out loud at least three times.
- Prepare 4 to 5 STAR stories and 2 questions to ask the interviewer.
- Research each company you are interviewing with: what it does and the role offered.
- Print copies of your resume; keep ID proof, college ID and documents ready.
- For online rounds: test your laptop, webcam, microphone, internet and charger the day before.
- Sleep well the night before. Do not start new topics in the last two days.

PrepSuccess prep guide · Practise these on PrepSuccess: Skill checks, Learn tasks and Interview prep.